Laporan
Uji Sertifikasi Kompetensi (USK)



Disusun oleh:

Nama	: Alvian Cahyo Pambudi
Kelas	: XII RPL 2
Progam Keahlian	: Pengembangan Perangkat Lunak dan Gim
Bidang Keahlian	: Rekayasa Perangkat Lunak

SMKN 64 JAKARTA

TAHUN PELAJARAN 2026/2027

Lembar Pengesahan

Laporan Uji Sertifikasi Kompetensi (USK) ini telah di setujui dan di sahkan : Di SMK NEGERI 64 JAKARTA

Jakarta,

Penguji / Asesor

Asesi

……

……

Ketua keahlian kompetensi RPL

……

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas rahmat dan karunia-Nya sehingga saya dapat menyelesaikan kegiatan Uji Sertifikasi Kompetensi (USK) dan penyusunan laporan "Certification App" dengan tepat waktu.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan USK. Proyek ini menjadi kesempatan untuk menerapkan keterampilan pengembangan perangkat lunak melalui aplikasi pengelolaan data peserta dan skema sertifikasi.

Saya mengucapkan terima kasih kepada guru dan pihak sekolah atas bimbingan serta dukungannya, serta kepada Bapak … selaku asesor yang telah memberikan arahan dan penilaian selama ujian.

Saya menyadari laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat saya harapkan, serta semoga laporan ini dapat memberikan manfaat.

Jakarta, ……

Hormat saya,

NIS: ……..

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat membuat kebutuhan terhadap sistem pengelolaan data yang terstruktur dan mudah digunakan menjadi semakin penting. Pengelolaan data yang masih dilakukan secara manual dapat menyebabkan berbagai kendala, seperti kesulitan dalam pencarian data, kemungkinan terjadinya kesalahan pencatatan, serta kurang efisiennya proses pengelolaan dan penyimpanan informasi.

Dalam kegiatan sertifikasi dan pelatihan, terdapat berbagai data yang perlu dikelola, seperti data peserta, data skema sertifikasi, serta informasi terkait pelatihan yang diikuti oleh peserta. Oleh karena itu, diperlukan sebuah aplikasi yang dapat membantu mengelola data tersebut secara lebih terorganisir.

Berdasarkan kebutuhan tersebut, pada kegiatan Uji Sertifikasi Kompetensi (USK) ini dibuat sebuah aplikasi bernama Certification App. Aplikasi ini dirancang untuk membantu proses pengelolaan data peserta pelatihan dan skema sertifikasi. Aplikasi dikembangkan dengan memanfaatkan Laravel sebagai REST API pada sisi backend dan React dengan TypeScript pada sisi frontend.

Melalui proyek ini, penulis dapat menerapkan pengetahuan dan keterampilan yang telah dipelajari selama proses pembelajaran, khususnya dalam pengembangan aplikasi berbasis web, pembuatan REST API, pengelolaan database, serta integrasi antara frontend dan backend.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam pembuatan aplikasi ini adalah:

1. Bagaimana membuat aplikasi yang dapat membantu mengelola data peserta pelatihan?
2. Bagaimana mengelola data skema sertifikasi secara terstruktur?
3. Bagaimana membuat sistem CRUD untuk mengelola data melalui aplikasi?

4. Bagaimana menghubungkan frontend dengan backend menggunakan REST API?
5. Bagaimana membuat proses pencarian data agar pengguna dapat menemukan data yang dibutuhkan dengan lebih mudah?

1.3 Tujuan

Tujuan dari pembuatan Certification App dalam kegiatan Uji Sertifikasi Kompetensi ini adalah:

1. Membuat aplikasi berbasis web untuk membantu pengelolaan data sertifikasi dan pelatihan.
2. Membuat sistem pengelolaan data peserta secara terstruktur.
3. Membuat sistem pengelolaan data skema sertifikasi.
4. Menerapkan konsep CRUD dalam pengembangan aplikasi.
5. Menerapkan penggunaan REST API sebagai penghubung antara frontend dan backend.
6. Menerapkan React dan TypeScript dalam pembuatan antarmuka aplikasi.
7. Menerapkan Laravel dalam pembuatan backend dan pengelolaan data.
8. Meningkatkan kemampuan penulis dalam mengembangkan aplikasi berbasis web.

1.4 Manfaat

1.4.1 Bagi Penulis

1. Meningkatkan kemampuan dalam mengembangkan aplikasi berbasis web.
2. Menerapkan ilmu yang telah dipelajari selama pembelajaran di sekolah.
3. Menambah pengalaman dalam menggunakan Laravel, React, TypeScript, dan REST API.
4. Melatih kemampuan dalam merancang dan mengembangkan sebuah aplikasi dari awal hingga dapat digunakan.

1.4.2 Bagi Pengguna

1. Mempermudah proses pengelolaan data peserta.

2. Mempermudah pengelolaan data skema sertifikasi.
3. Mempermudah pencarian data yang dibutuhkan.
4. Membantu menyimpan data secara lebih terstruktur.

1.4.3 Bagi Sekolah

1. Menjadi salah satu penerapan kompetensi siswa dalam bidang Rekayasa Perangkat Lunak.
2. Menjadi dokumentasi hasil pelaksanaan Uji Sertifikasi Kompetensi.
3. Menunjukkan kemampuan siswa dalam menerapkan teknologi pengembangan perangkat lunak.

1.5 Batasan Masalah

Agar pembahasan tidak terlalu luas, proyek **Certification App** memiliki beberapa batasan sebagai berikut:

1. Aplikasi berfokus pada pengelolaan data yang berkaitan dengan peserta dan skema sertifikasi.
2. Sistem memiliki fitur untuk menampilkan, menambahkan, mengubah, dan menghapus data.
3. Data disimpan dan dikelola melalui database pada sisi backend.
4. Backend aplikasi menggunakan Laravel dan menyediakan REST API.
5. Frontend aplikasi menggunakan React dan TypeScript.
6. Aplikasi dibuat sebagai aplikasi web dan belum mencakup pengembangan aplikasi mobile.
7. Pembahasan laporan berfokus pada proses pengembangan aplikasi dan fitur yang dibuat selama pelaksanaan USK.

LANDASAN TEORI

2.1 Uji Sertifikasi Kompetensi

Uji Sertifikasi Kompetensi (USK) merupakan kegiatan yang bertujuan untuk mengukur kemampuan peserta didik dalam menerapkan kompetensi yang telah dipelajari selama proses pendidikan. Dalam pelaksanaannya, peserta diberikan tugas atau proyek yang sesuai dengan bidang keahlian masing-masing.

Pada bidang Rekayasa Perangkat Lunak, salah satu bentuk penerapan kompetensi tersebut adalah pengembangan aplikasi. Melalui proyek **Certification App**, penulis menerapkan kemampuan dalam analisis kebutuhan, perancangan database, pembuatan backend, pembuatan frontend, serta pengujian aplikasi.

2.2 Aplikasi Berbasis Web

Aplikasi berbasis web merupakan aplikasi yang dapat diakses melalui web browser dan umumnya menggunakan teknologi frontend serta backend untuk menjalankan fungsinya.

Pada proyek ini, aplikasi berbasis web digunakan sebagai media untuk mengelola data peserta dan skema sertifikasi. Pengguna dapat melakukan proses pengelolaan data melalui antarmuka yang tersedia.

2.3 Laravel

Laravel merupakan framework PHP yang digunakan untuk membangun aplikasi web. Laravel menyediakan berbagai fitur yang membantu proses pengembangan aplikasi, seperti routing, migration, model, controller, validasi, dan pengelolaan database.

Pada proyek **Certification App**, Laravel digunakan sebagai bagian backend untuk mengelola data dan menyediakan REST API yang digunakan oleh frontend.

2.4 REST API

REST API atau Representational State Transfer Application Programming Interface merupakan suatu mekanisme yang memungkinkan aplikasi saling berkomunikasi menggunakan protokol HTTP.

Pada aplikasi ini, REST API digunakan sebagai penghubung antara frontend React dan backend Laravel. Beberapa metode HTTP yang digunakan antara lain:

- **GET** untuk mengambil data.
- **POST** untuk menambahkan data.
- **PUT/PATCH** untuk memperbarui data.
- **DELETE** untuk menghapus data.

2.5 React

React merupakan library JavaScript yang digunakan untuk membangun antarmuka pengguna. React menggunakan konsep component sehingga bagian-bagian antarmuka dapat dibuat secara terpisah dan digunakan kembali.

Pada proyek ini, React digunakan untuk membangun halaman frontend dan mengelola interaksi pengguna dengan aplikasi.

2.6 TypeScript

TypeScript merupakan bahasa pemrograman yang merupakan pengembangan dari JavaScript dengan tambahan sistem tipe data.

TypeScript digunakan pada bagian frontend proyek **Certification App** untuk membantu menentukan tipe data yang digunakan, sehingga struktur data dari API dapat dikelola dengan lebih terstruktur.

2.7 Database

Database merupakan tempat penyimpanan data yang digunakan oleh aplikasi. Database memungkinkan data disimpan secara terstruktur dan dapat diakses maupun diubah sesuai kebutuhan aplikasi.

Pada **Certification App**, database digunakan untuk menyimpan data seperti peserta dan skema sertifikasi serta hubungan antar data tersebut.

2.8 CRUD

CRUD merupakan singkatan dari:

- **C**reate – menambahkan data.
- **R**ead – menampilkan atau membaca data.
- **U**pdate – mengubah data.
- **D**elete – menghapus data.

Konsep CRUD merupakan salah satu fungsi utama yang diterapkan pada **Certification App** untuk mengelola data.

ANALISIS DAN PERANCANGAN

3.1 Analisis Kebutuhan

Sebelum melakukan pengembangan aplikasi, dilakukan analisis terhadap kebutuhan sistem. Analisis dilakukan untuk menentukan fitur dan data yang diperlukan agar aplikasi dapat digunakan sesuai dengan tujuan yang telah ditentukan.

Berdasarkan hasil analisis, aplikasi membutuhkan beberapa fungsi utama, yaitu:

1. Menampilkan data peserta.
2. Menambahkan data peserta.
3. Mengubah data peserta.
4. Menghapus data peserta.
5. Menampilkan data skema sertifikasi.
6. Menambahkan data skema sertifikasi.
7. Mengubah data skema sertifikasi.
8. Menghapus data skema sertifikasi.
9. Menghubungkan peserta dengan skema sertifikasi.
10. Melakukan pencarian data.

3.2 Kebutuhan Perangkat Keras

Perangkat keras yang digunakan dalam proses pengembangan aplikasi antara lain:

No.	Perangkat	Keterangan
1	Laptop	Digunakan untuk proses pengembangan aplikasi

2	Keyboard	Digunakan untuk proses pengetikan kode program
3	Mouse/Touch pad	Digunakan untuk mengoperasikan aplikasi
4	Smartphone	Digunakan jika diperlukan untuk pengujian tampilan

3.3 Kebutuhan Perangkat Lunak

Perangkat lunak yang digunakan dalam pengembangan aplikasi antara lain:

No	Perangkat Lunak	Fungsi
1	Visual Studio Code	Menulis dan mengelola kode program
2	Laravel	Mengembangkan backend dan REST API
3	React	Mengembangkan frontend
4	TypeScript	Menentukan tipe data pada frontend
5	PHP	Menjalankan backend Laravel

6	Node.js	Menjalankan kebutuhan pengembangan frontend
7	Database Management System	Menyimpan dan mengelola data
8	Git	Mengelola versi kode program
9	Web Browser	Menjalankan dan menguji aplikasi

3.4 Perancangan Sistem

Sistem pada **Certification App** dibagi menjadi dua bagian utama, yaitu frontend dan backend.

3.4.1 Backend

Backend berfungsi untuk mengelola proses bisnis, database, validasi data, serta menyediakan REST API yang dapat digunakan oleh frontend.

Laravel digunakan sebagai framework backend. Data yang diterima dari frontend akan diproses oleh API kemudian disimpan atau diambil dari database.

3.4.2 Frontend

Frontend merupakan bagian aplikasi yang berinteraksi langsung dengan pengguna. React digunakan untuk membuat halaman dan komponen antarmuka.

Frontend mengirimkan request ke REST API yang dibuat menggunakan Laravel. Data yang diterima dari API kemudian ditampilkan pada halaman aplikasi.

3.5 Perancangan Database

Database digunakan untuk menyimpan data yang dibutuhkan oleh aplikasi. Salah satu data utama yang digunakan adalah data **skema sertifikasi** dan **peserta**.

Data skema sertifikasi digunakan untuk menyimpan informasi mengenai skema yang tersedia, sedangkan data peserta digunakan untuk menyimpan informasi peserta yang mengikuti sertifikasi atau pelatihan.

Hubungan antara data peserta dan skema sertifikasi digunakan agar setiap peserta dapat dikaitkan dengan skema sertifikasi yang dipilih.

3.6 Perancangan Fitur

3.6.1 Fitur Data Skema Sertifikasi

Fitur ini digunakan untuk mengelola data skema sertifikasi. Informasi yang dapat dikelola antara lain:

- Kode skema.
- Nama skema.
- Deskripsi skema.

Pengguna dapat melihat data, menambahkan data baru, mengubah data, dan menghapus data skema.

3.6.2 Fitur Data Peserta

Fitur peserta digunakan untuk mengelola informasi peserta. Data yang dapat dikelola antara lain:

- Nama lengkap.
- Email.
- Nomor telepon.

- Tanggal lahir.
- Jenis kelamin.
- Alamat.
- Skema sertifikasi.

3.6.3 Fitur Pencarian

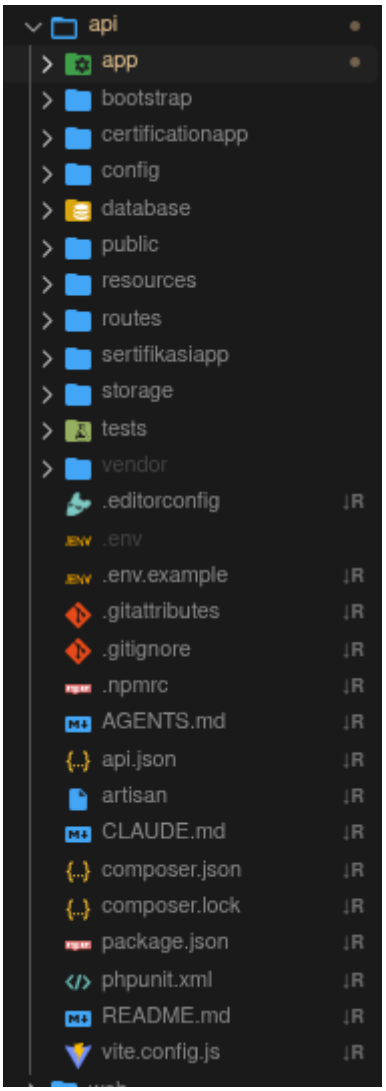
Fitur pencarian digunakan untuk membantu pengguna menemukan data tertentu tanpa harus mencari secara manual dari seluruh data yang tersedia.

Fitur ini membuat proses pengelolaan data menjadi lebih cepat dan efisien.

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Backend

Struktur folder backend laravel



Routes/API Laravel

```php
Route::prefix('/v1')->group(function () {
    Route::prefix('/auth')->group(function () {
        Route::post('/register', [AuthController::class, 'register']);
        Route::post('/login', [AuthController::class, 'login']);
        Route::middleware('auth:sanctum')->group(function () {
            Route::post('/logout', [AuthController::class, 'logout']);
        });
    });

    Route::middleware('auth:sanctum')->group(function() {
    Route::get('/candidates', [CandidateController::class, 'index']);
    Route::get('/candidates/{id}', [CandidateController::class, 'show']);
    Route::post('/candidates', [CandidateController::class, 'store']);
    Route::put('/candidates/{id}', [CandidateController::class, 'update']);
    Route::delete('/candidates/{id}', [CandidateController::class, 'destroy']);

    Route::get('/schemes', [SchemeController::class, 'index']);
    Route::get('/schemes/{id}', [SchemeController::class, 'show']);
    Route::post('/schemes', [SchemeController::class, 'store']);
    Route::put('/schemes/{id}', [SchemeController::class, 'update']);
    Route::delete('/schemes/{id}', [SchemeController::class, 'destroy']);

    Route::get('/users', [UserController::class, 'index']);
    Route::get('/users/{id}', [UserController::class, 'show']);
    Route::post('/users', [UserController::class, 'store']);
    Route::put('/users/{id}', [UserController::class, 'update']);
    Route::delete('/users/{id}', [UserController::class, 'destroy']);
});
});
```

Scheme Controller

```php
class SchemeController extends Controller
{
    public function index() {
        $schemes = CertificationScheme::all();

        return response()->json([
            'status' => 'success',
            'message' => 'Get all scheme successfully',
            'data' => [
                'schemes' => SchemeResource::collection($schemes),
            ]
        ]);
    }

    public function show(string $id) {
        $scheme = CertificationScheme::where('id', $id)->first();

        if (!$scheme) {
            return response()->json([
                'status' => 'error',
                'message' => 'Scheme not found'
            ], 404);
        }

        return response()->json([
            'status' => 'success',
            'message' => 'Get scheme successfully',
            'data' => [
                'scheme' => new SchemeResource($scheme)
            ]
        ]);
    }

    public function store(Request $request) {
        $validated = $request->validate([
            'scheme_code' => 'required',
            'scheme_name' => 'required',
            'description' => 'required'
        ]);

        $scheme = CertificationScheme::create([
            'scheme_code' => $validated['scheme_code'],
            'scheme_name' => $validated['scheme_name'],
            'description' => $validated['description'],
        ]);

        return response()->json([
            'status' => 'success',
            'message' => 'Added scheme successfully',
            'data' => [
                'scheme' => $scheme
            ]
        ], 201);
    }
```

```php
public function update(Request $request, string $id) {
    $validated = $request->validate([
        'scheme_code' => 'nullable',
        'scheme_name' => 'nullable',
        'description' => 'nullable'
    ]);

    $scheme = CertificationScheme::where('id', $id)->first();

    if (!$scheme) {
        return response()->json([
            'status' => 'error',
            'message' => 'Scheme not found'
        ], 404);
    }

    $scheme->update($validated);

    return response()->json([
        'status' => 'success',
        'message' => 'Updated scheme successfully',
        'data' => [
            'scheme' => $scheme
        ]
    ], 200);
}

public function destroy(string $id) {
    $scheme = CertificationScheme::where('id', $id)->first();

    if (!$scheme) {
        return response()->json([
            'status' => 'error',
            'message' => 'Scheme not found'
        ], 404);
    }

    $scheme->delete();

    return response()->json([
        'status' => 'success',
        'message' => 'Delete scheme successfully'
    ]);
}
```

Candidate Controller

```php
class CandidateController extends Controller
{
    public function index() {
        $candidates = CertificationCandidate::all();

        return response()->json([
            'status' => 'success',
            'message' => 'Get all candidate successfully',
            'data' => [
                'candidates' => CandidateResource::collection($candidates)
            ]
        ]);
    }

    public function show(Request $request, string $id) {
        $candidate = CertificationCandidate::where('id', $id)->first();

        if (!$candidate) {
            return response()->json([
                'status' => 'error',
                'message' => 'Candidate not found'
            ], 404);
        }

        return response()->json([
            'status' => 'success',
            'message' => 'Get candidate successfully',
            'data' => [
                'candidate' => new CandidateResource($candidate)
            ]
        ]);
    }
```

```php
public function store(Request $request) {
    $validated = $request->validate([
        'full_name' => 'required',
        'email' => 'required',
        'phone_number' => 'required',
        'born_date' => 'required|date',
        'gender' => 'required',
        'address' => 'required',
        'certification_scheme_id' => 'required|exists:certification_schemes,id'
    ]);

    $scheme = CertificationScheme::where('id', $validated['certification_scheme_id'])->first();

    $candidate = CertificationCandidate::create([
        'candidate_number' => rand(100000, 999999),
        'full_name' => $validated['full_name'],
        'email' => $validated['email'],
        'phone_number' => $validated['phone_number'],
        'born_date' => $validated['born_date'],
        'gender' => $validated['gender'],
        'address' => $validated['address'],
        'certification_scheme_id' => $validated['certification_scheme_id'],
    ]);

    return response()->json([
        'status' => 'success',
        'message' => 'Added candidate successfully',
        'data' => [
            'candidate' => new CandidateResource($candidate)
        ]
    ], 201);
}
```

```php
    public function update(Request $request, string $id) {
        $validated = $request->validate([
            'full_name' => 'nullable',
            'email' => 'nullable',
            'phone_number' => 'nullable',
            'born_date' => 'nullable|date',
            'gender' => 'nullable',
            'address' => 'nullable',
            'certification_scheme_id' => 'nullable|exists:certification_schemes,id'
        ]);

        $candidate = CertificationCandidate::where('id', $id)->first();

        if (!$candidate) {
            return response()->json([
                'status' => 'error',
                'message' => 'Candidate not found'
            ], 404);
        }

        $candidate->update($validated);

        return response()->json([
            'status' => 'success',
            'message' => 'Updated candidate successfully',
            'data' => [
                'candidate' => new CandidateResource($candidate)
            ]
        ], 200);
    }

    public function destroy(string $id) {
        $candidate = CertificationCandidate::where('id', $id)->first();

        if (!$candidate) {
            return response()->json([
                'status' => 'error',
                'message' => 'Candidate not found'
            ], 404);
        }

        $candidate->delete();

        return response()->json([
            'status' => 'success',
            'message' => 'Delete Candidate successfully'
        ]);
    }
}
```

Model and Relationship

api > app > Models > CertificationCandidate.php > ...

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Model;

#[Fillable([
    'candidate_number',
    'full_name',
    'email',
    'phone_number',
    'born_date',
    'gender',
    'address',
    'certification_scheme_id'
])]
class CertificationCandidate extends Model
{
    public function certificationScheme() {
        return $this->belongsTo(CertificationScheme::class);
    }

    protected $casts = [
        'created_at' => 'datetime: d M Y: H:i',
        'updated_at' => 'datetime: d M Y: H:i'
    ];
}
```

api > app > Models > CertificationScheme.php > ...

```php
<?php

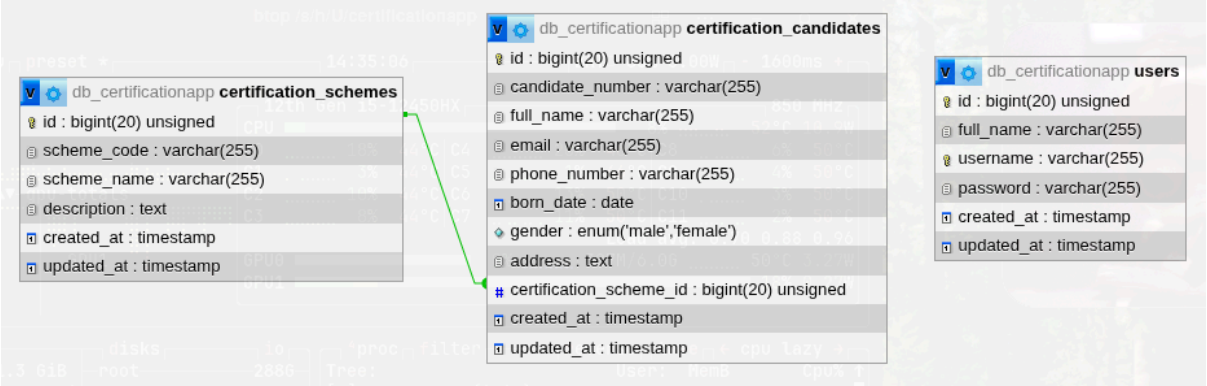
namespace App\Models;

use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Model;

#[Fillable([
    'scheme_code',
    'scheme_name',
    'description'
])]
class CertificationScheme extends Model
{
    public function certificationCandidate() {
        return $this->hasMany(CertificationCandidate::class);
    }

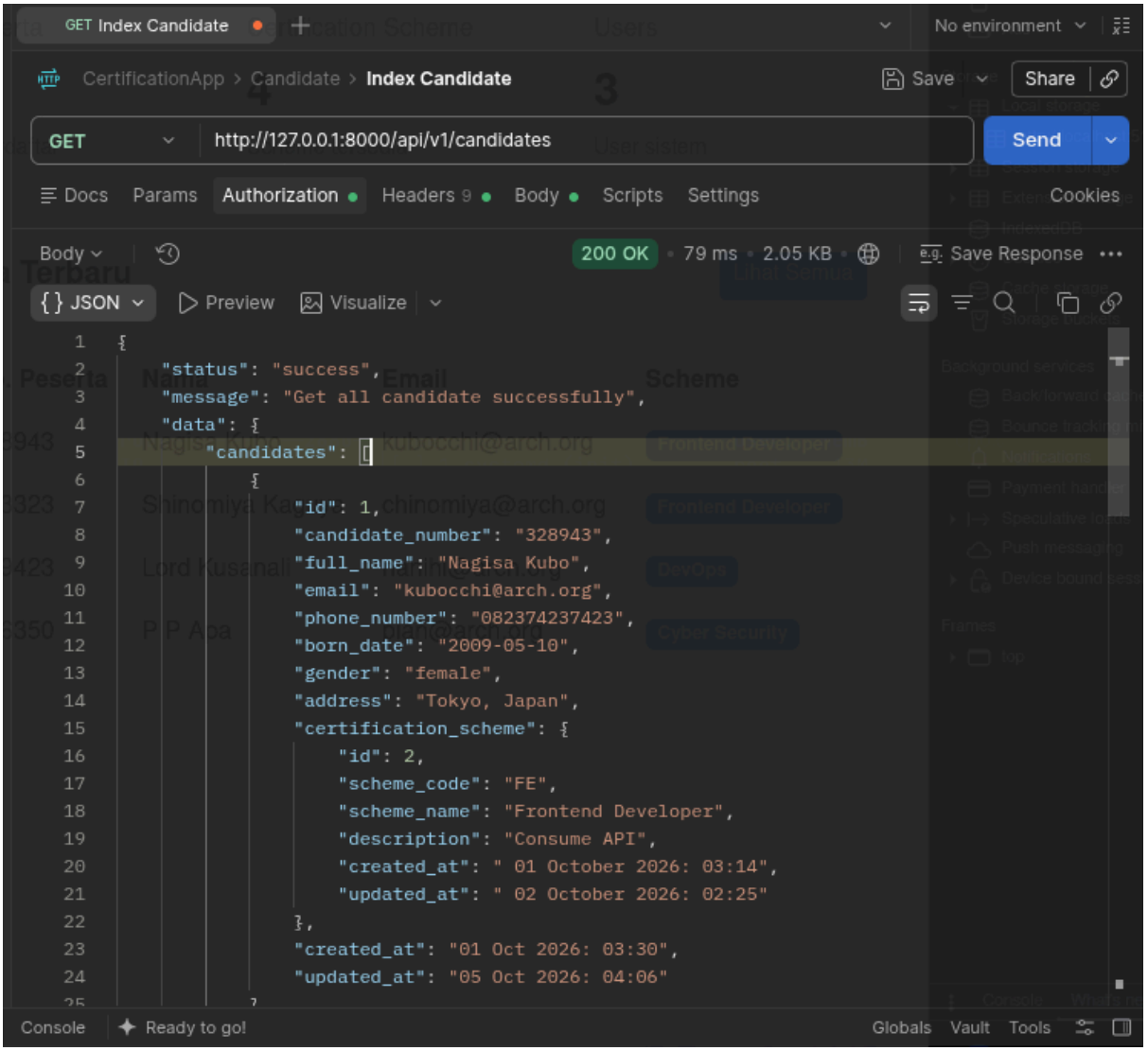
    protected $casts = [
        'created_at' => 'datetime: d F Y: H:i',
        'updated_at' => 'datetime: d F Y: H:i'
    ];
}

```

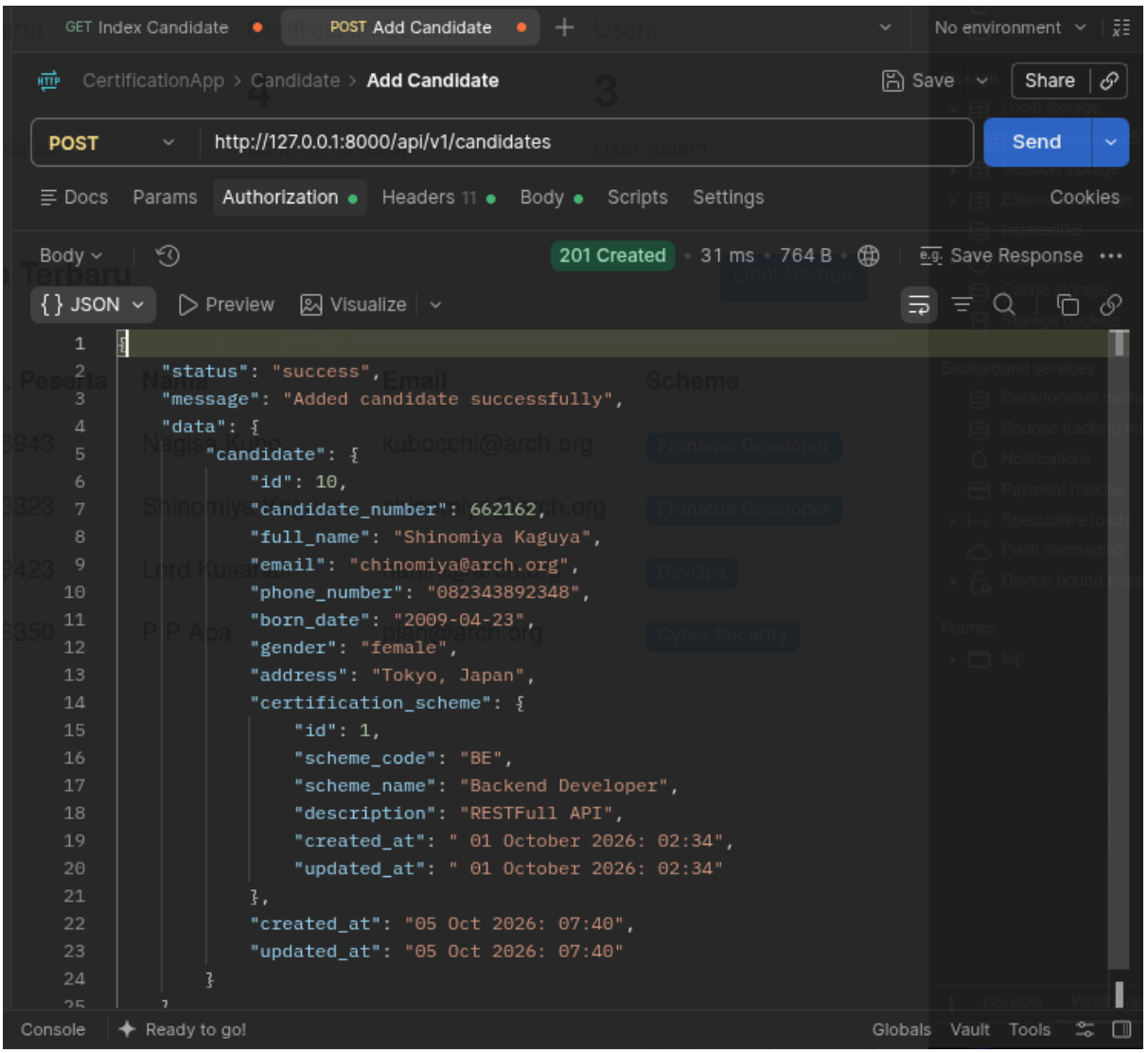


4.2 Implementasi RESTAPI

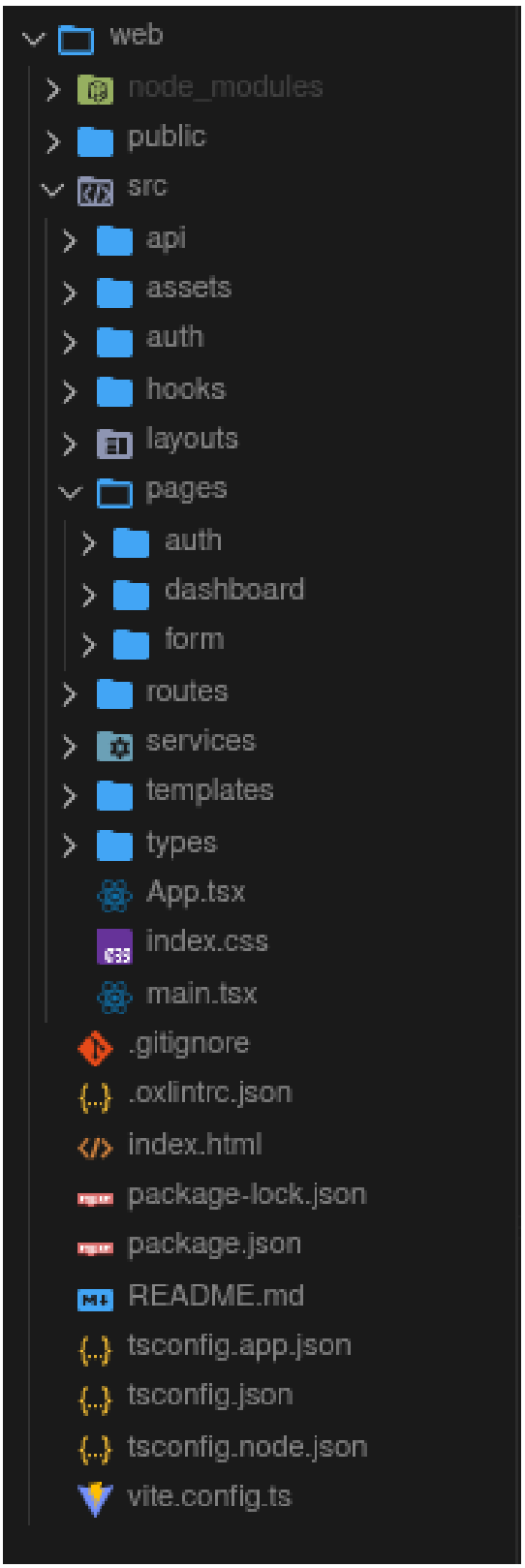
Pengujian endpoint GET Candidate



Pengujian endpoint POST Candidate



4.3 Implementasi Frontend



Service/API React

web > src > api > TS api.ts > default

```ts
import axios from "axios";

export const BASE_URL = "http://127.0.0.1:8000/api/v1";

const api = axios.create({
    baseURL: BASE_URL
});

api.interceptors.request.use(
    (config) => {
        const token = localStorage.getItem("token");
        if (token) {
            config.headers.Authorization = `Bearer ${token}`
        } return config;
    }
)

export default api;
```

web > src > services > TS candidateService.ts > destroyCandidate

```ts
import api from "../api/api"
import type { CandidatePayload, CandidateResponse, CandidatesResponse } from "../types/candidate.type";
import type { ApiResponse } from "../types/response.type";

export const indexCandidate = () => {
    return api.get<ApiResponse<CandidatesResponse>>('/candidates');
}

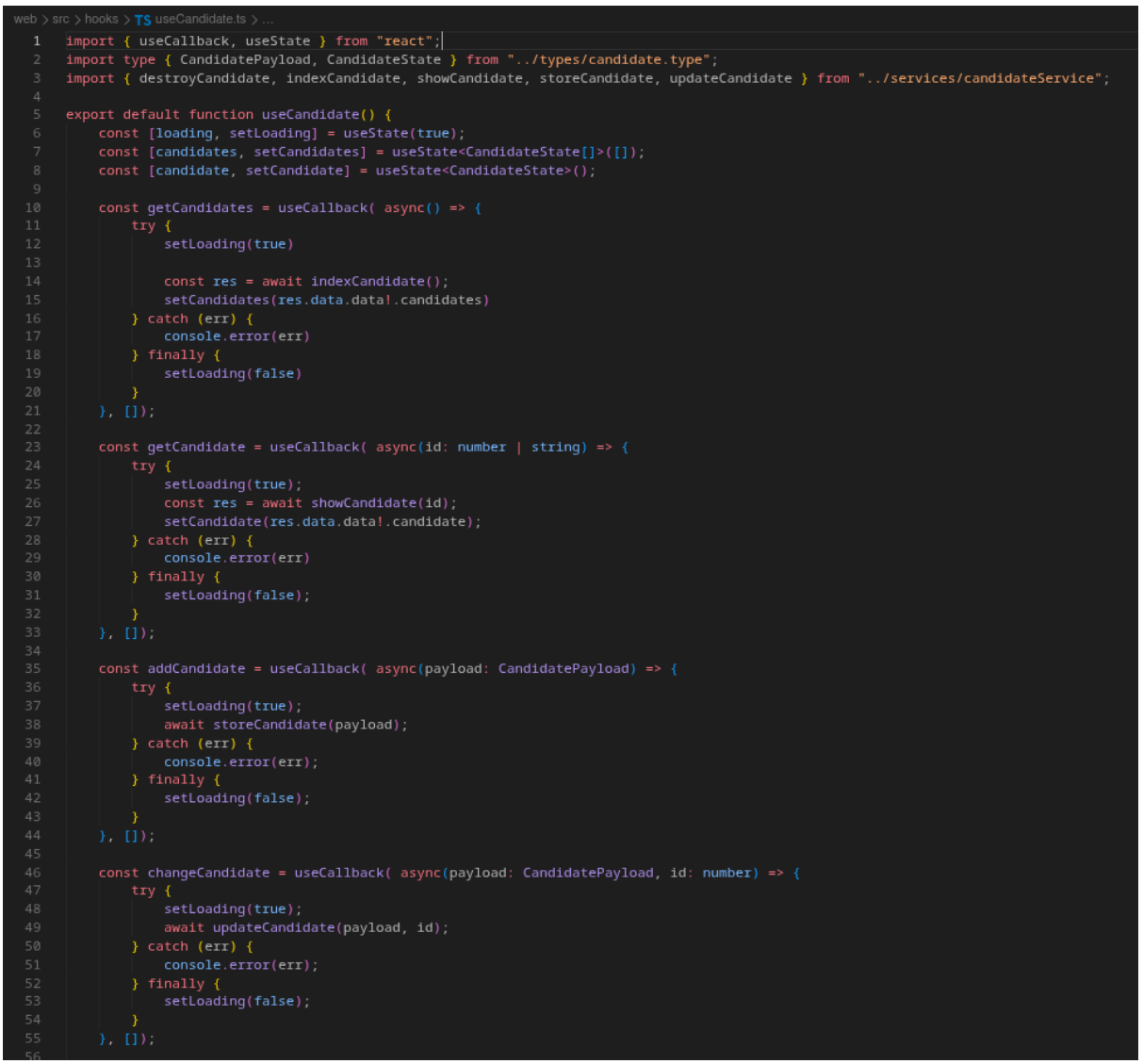
export const showCandidate = (id: number | string) => {
    return api.get<ApiResponse<CandidateResponse>>(`/candidates/${id}`);
}

export const storeCandidate = (payload: CandidatePayload) => {
    return api.post('/candidates', payload);
}

export const updateCandidate = (payload: CandidatePayload, id: number) => {
    return api.put(`/candidates/${id}`, payload);
}

export const destroyCandidate = (id: number | string) => {
    return api.delete(`/candidates/${id}`)
}
```

Custom Hook useCandidate



web > src > hooks > TS useCandidate.ts > ...

```ts
import { useCallback, useState } from "react";
import type { CandidatePayload, CandidateState } from "../types/candidate.type";
import { destroyCandidate, indexCandidate, showCandidate, storeCandidate, updateCandidate } from "../services/candidateService";

export default function useCandidate() {
    const [loading, setLoading] = useState(true);
    const [candidates, setCandidates] = useState<CandidateState[]>([]);
    const [candidate, setCandidate] = useState<CandidateState>();

    const getCandidates = useCallback( async() => {
        try {
            setLoading(true)

            const res = await indexCandidate();
            setCandidates(res.data.data!.candidates)
        } catch (err) {
            console.error(err)
        } finally {
            setLoading(false)
        }
    }, []);

    const getCandidate = useCallback( async(id: number | string) => {
        try {
            setLoading(true);
            const res = await showCandidate(id);
            setCandidate(res.data.data!.candidate);
        } catch (err) {
            console.error(err)
        } finally {
            setLoading(false);
        }
    }, []);

    const addCandidate = useCallback( async(payload: CandidatePayload) => {
        try {
            setLoading(true);
            await storeCandidate(payload);
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    }, []);

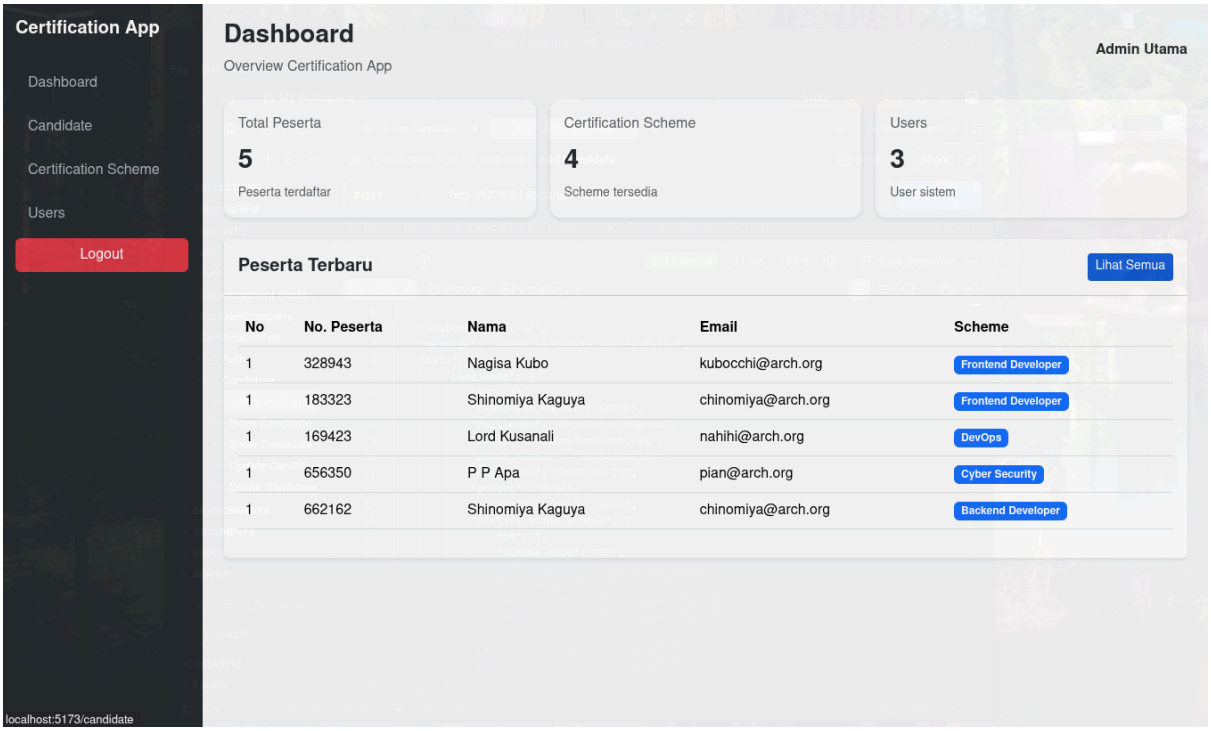
    const changeCandidate = useCallback( async(payload: CandidatePayload, id: number) => {
        try {
            setLoading(true);
            await updateCandidate(payload, id);
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    }, []);
```

```
    const deleteCandidate = useCallback( async(id: number) => {
        try {
            setLoading(true);
            await destroyCandidate(id);
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    }, []);

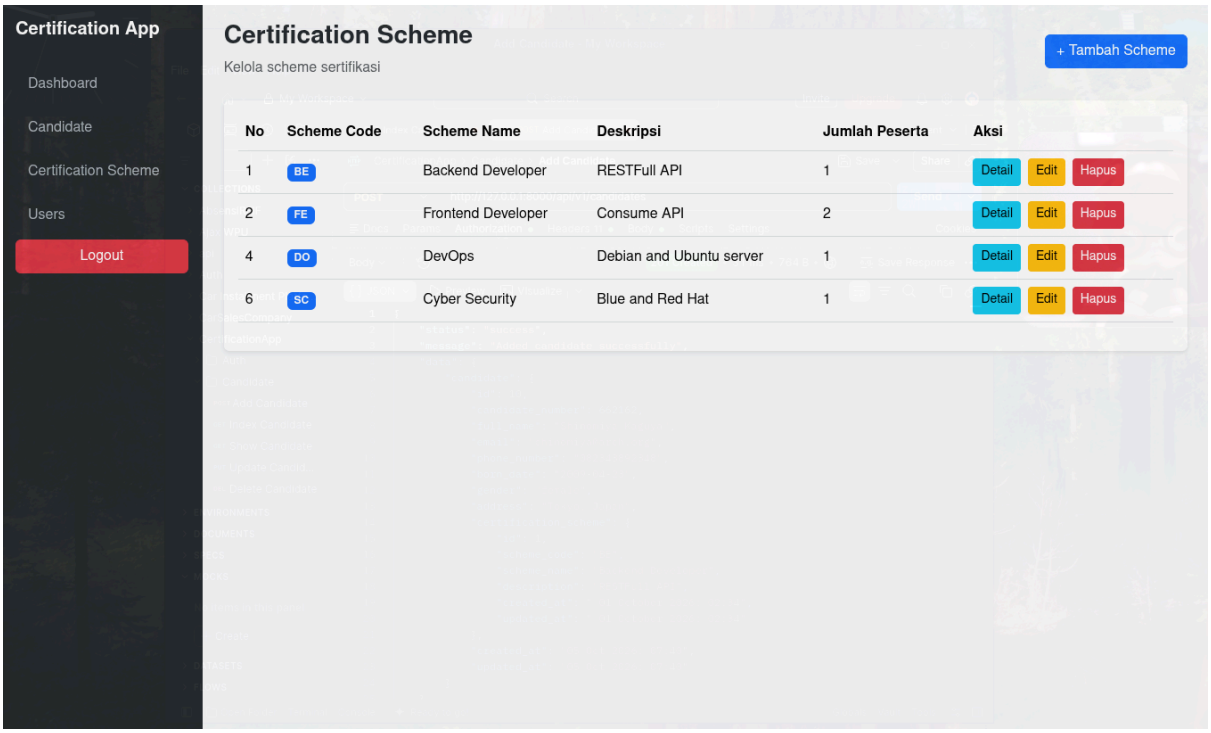
    return {
        loading,
        candidates,
        candidate,
        getCandidates,
        getCandidate,
        addCandidate,
        changeCandidate,
        deleteCandidate
    }
}
```

4.4 Implementasi Antarmuka

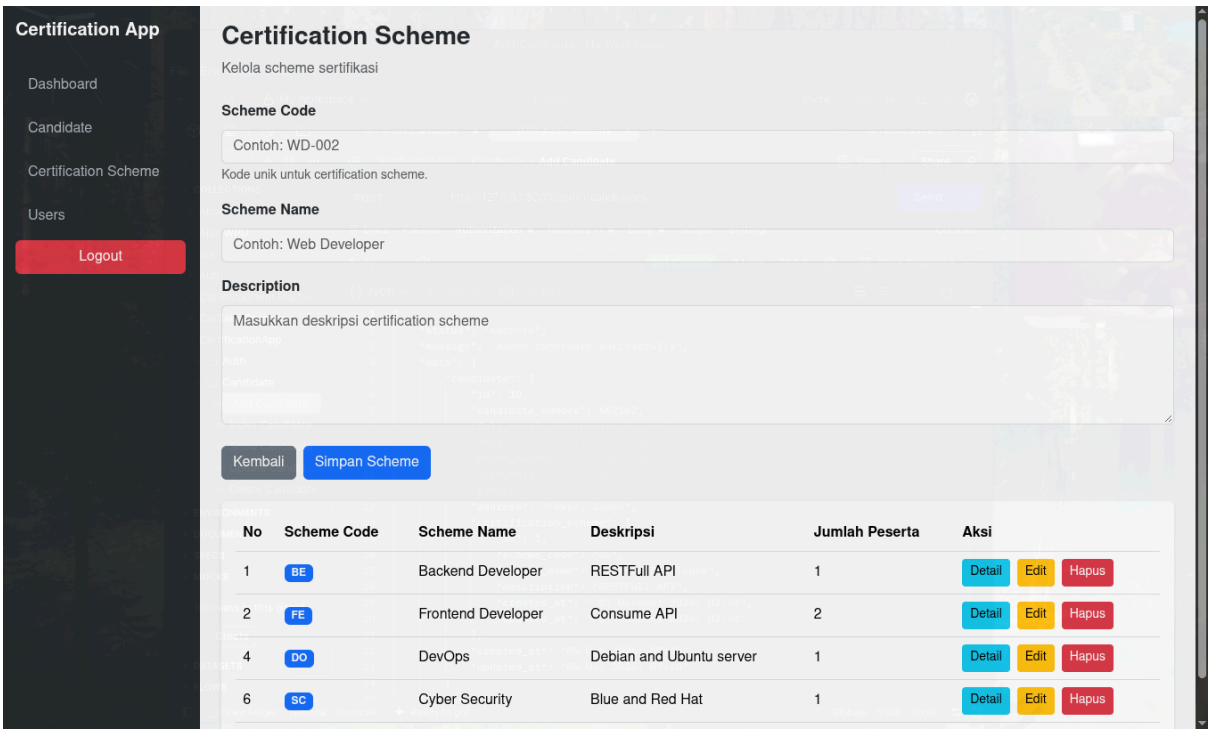
Dashboard



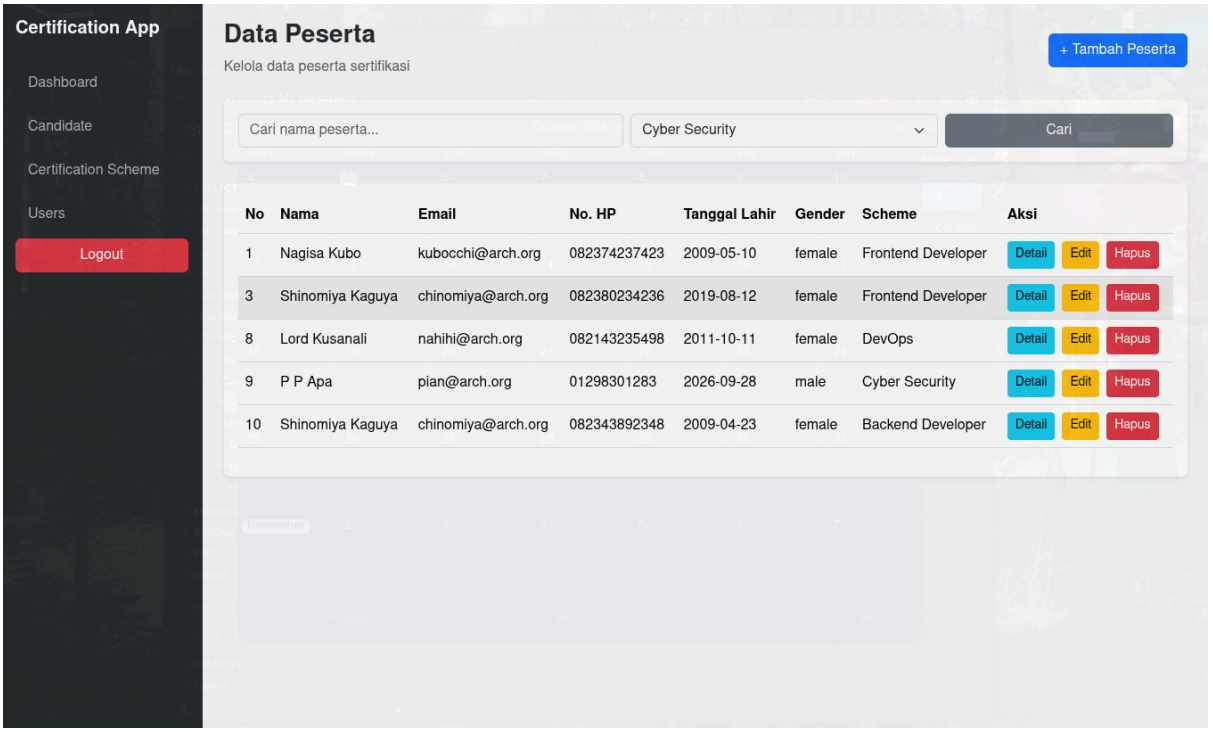
Halaman Scheme



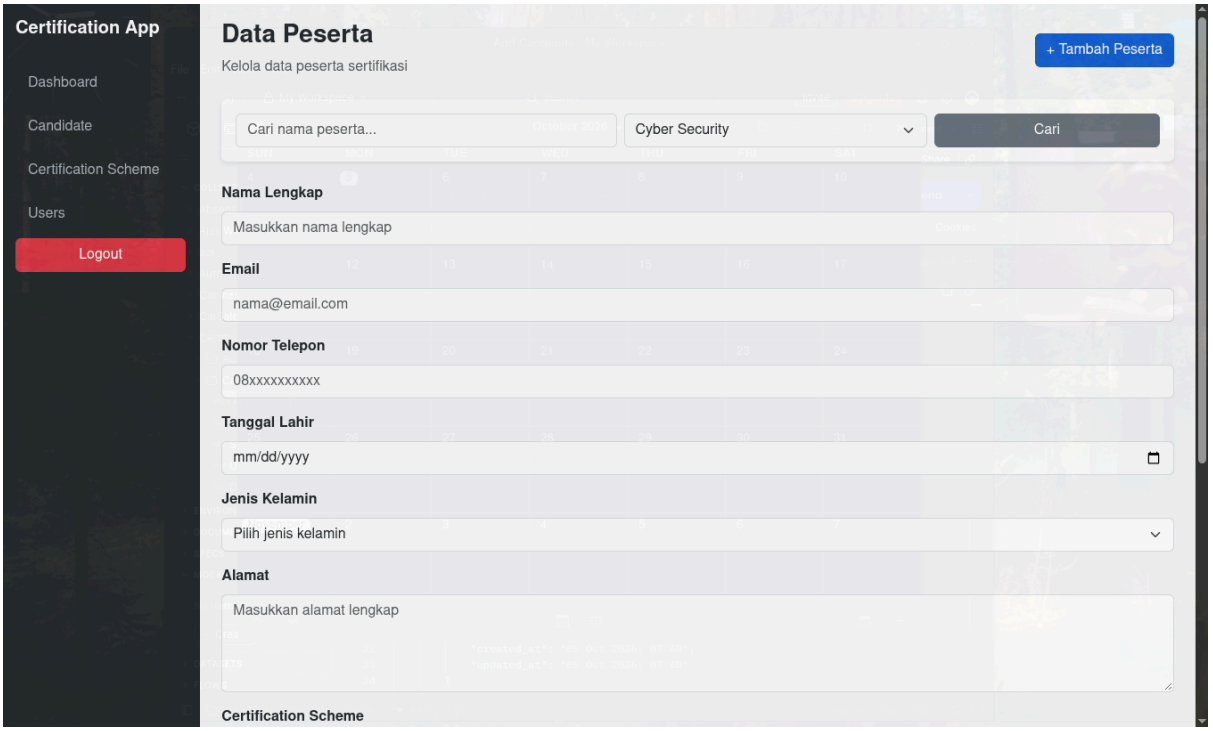
Form tambah/edit Scheme



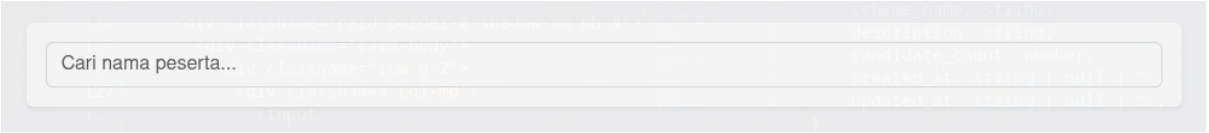
Halaman Candidate



Form tambah/edit Candidate

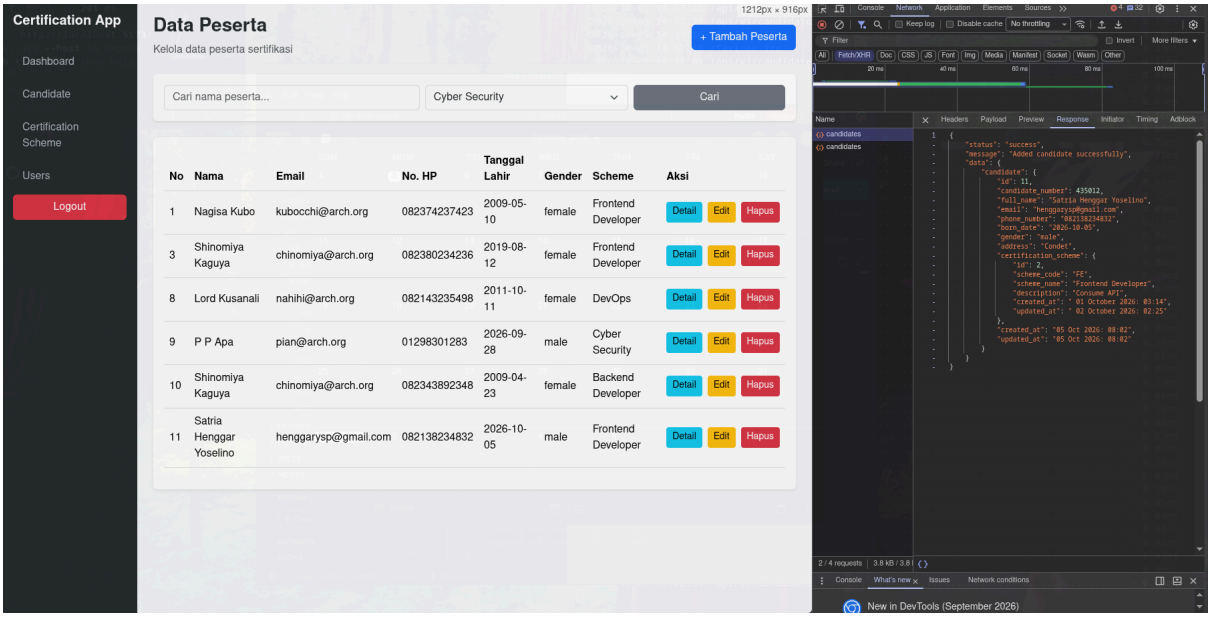


Fitur Search

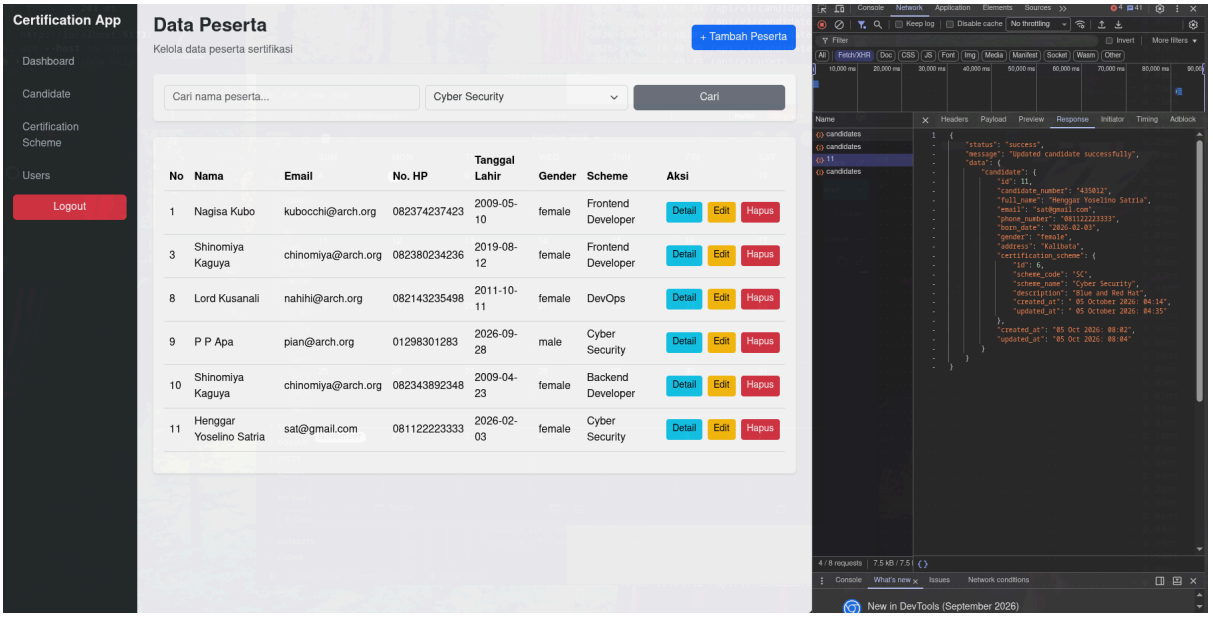


4.5 Pengujian Aplikasi

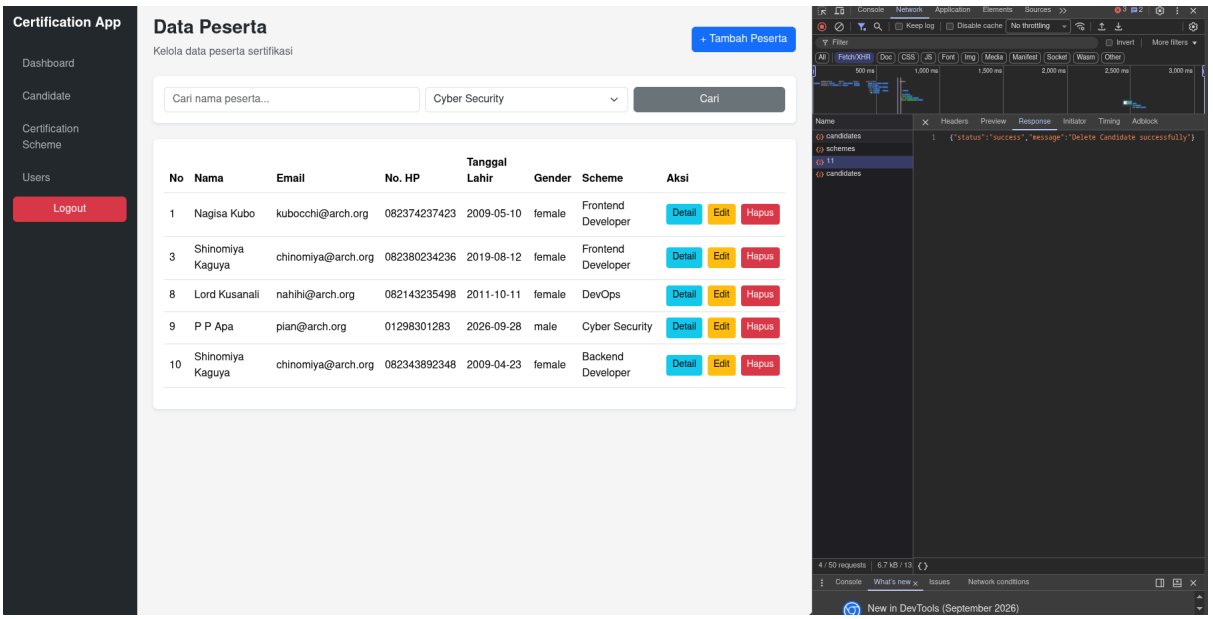
Pengujian tambah data



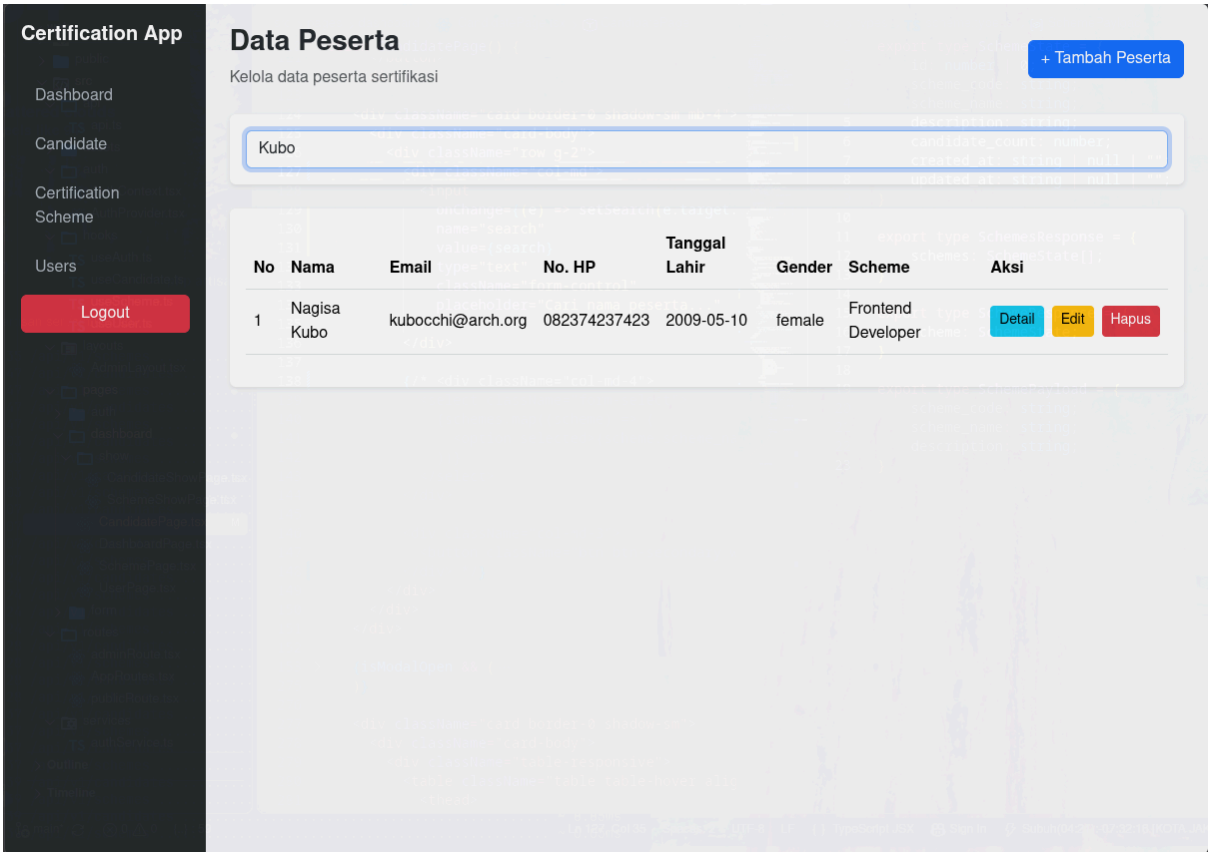
Pengujian edit data



Pengujian hapus data



Pengujian search



PENUTUP

5.1 Kesimpulan

Berdasarkan proses perancangan, pembuatan, dan pengujian **Certification App**, dapat disimpulkan bahwa:

1. **Certification App** berhasil dibuat sebagai aplikasi berbasis web untuk membantu proses pengelolaan data peserta dan skema sertifikasi.
2. Laravel berhasil digunakan sebagai backend untuk mengelola database dan menyediakan REST API.
3. React dan TypeScript berhasil digunakan untuk membangun frontend dan antarmuka aplikasi.
4. Sistem dapat menjalankan fungsi CRUD untuk mengelola data.
5. Data peserta dapat dihubungkan dengan skema sertifikasi yang tersedia.
6. Fitur pencarian dapat membantu pengguna menemukan data dengan lebih mudah.
7. Pengembangan proyek ini memberikan pengalaman kepada penulis dalam menerapkan konsep pengembangan aplikasi berbasis web, khususnya dalam integrasi frontend, backend, REST API, dan database.

5.2 Saran

Meskipun aplikasi telah dapat menjalankan fungsi utama, masih terdapat beberapa hal yang dapat dikembangkan pada tahap selanjutnya.

Beberapa pengembangan yang dapat dilakukan antara lain:

1. Menambahkan sistem autentikasi dan hak akses pengguna yang lebih lengkap.
2. Menambahkan validasi data yang lebih mendalam pada setiap form.
3. Mengembangkan fitur pencarian dan filter agar lebih fleksibel.
4. Menambahkan fitur export data ke dalam format seperti PDF atau Excel.

5. Meningkatkan desain antarmuka agar lebih responsif dan mudah digunakan.
6. Menambahkan fitur laporan untuk menampilkan data peserta berdasarkan skema sertifikasi.
7. Meningkatkan keamanan API dan pengelolaan data pengguna.
8. Mengembangkan aplikasi agar dapat digunakan pada perangkat mobile.

Dengan adanya pengembangan tersebut, diharapkan **Certification App** dapat memiliki fitur yang lebih lengkap serta memberikan manfaat yang lebih besar dalam proses pengelolaan data sertifikasi dan pelatihan.